Generative AI — Assignment 3: Group Mini-Project

Stage 1: Idea Submission

ClauseCheck: An Agentic RAG System for Health Insurance Claim Adjudication

Programme: AI & Data Science, Jio Institute **Date:** 22 September 2026

1. Group Members

#	Name	ID
1	Gundeep Singh	gundeep.s27pgai@jioinstitute.edu.in
2	Aditya Bakhale	aditya.b27pgai@jioinstitute.edu.in
3	Raghavendra Prasad Shetty	raghavendraprasad.s27PGAI@jioinstitute.edu.in

2. Problem Statement

A retail health insurance policy wording typically runs to 50-80 pages. Whether a hospitalisation claim is payable, and for how much, depends on several clauses spread across that document: the pre-existing disease waiting period, ailment-specific waiting periods, permanent and temporary exclusions, room rent capping, procedure sub-limits and co-payment. Insurers word these clauses differently. The final payable amount follows from them arithmetically.

At present this is done manually. Claims staff read the policy wording, locate the applicable clauses and calculate the deduction by hand. Turnaround is slow, and rejection letters often reach the customer without any clause reference. Claim settlement remains the largest category of complaint received by the insurance ombudsman.

We propose a decision support system that takes a claim scenario and a policy, and returns a payable, not payable or partially payable verdict, along with the payable amount and the clauses it relied on, giving the insurer, section number and page for each.

Why this is a suitable topic for a RAG project:

1. The answer depends entirely on retrieval. The information needed is specific to one insurer's wording and one claim, so a model answering from its own knowledge will produce a plausible but incorrect waiting period. Retrieval is required for the output to be correct at all.
2. The output can be checked. Every verdict points to a clause on a page, so anyone can open the policy PDF and confirm whether the citation is right. This lets us measure grounding instead of assuming it.
3. A single claim needs several separate clause lookups and a calculation. One retrieve-and-answer call cannot cover this, which is why we are using an agent graph.
4. The data is public. Policy wordings are published on insurer websites, and the claim scenarios are written by us, so there is no privacy concern.
5. There is a clear business case: faster claim processing, fewer complaints escalated to the ombudsman, and a record of why each decision was made.

ClauseCheck is meant to support a human adjudicator, not to reject claims automatically. If the retrieved evidence is not sufficient, the system says so and refers the claim to a person.

3. Proposed Approach

3.1 Custom dataset

- Policy wordings: 4-5 retail indemnity health policy PDFs from 3-4 Indian insurers, downloaded from their product pages. We are keeping the set small so that we can check the system's citations against the source ourselves.
- Gold evaluation set: 20-25 claim scenarios written by us, each labelled with the expected verdict, the clause that governs it and the expected payable amount. We need this set in order to measure anything.

3.2 Ingestion and indexing

- Clause-aware chunking based on the headings in the document (Definitions, Waiting Periods, Exclusions, Table of Benefits) instead of fixed-size splitting, so that a chunk does not cut across two unrelated clauses. Each chunk keeps its section heading as a prefix, so the provisos attached to a clause stay with it.
- Metadata per chunk: insurer, product name, clause type (waiting period, exclusion, sub-limit, definition, co-pay), section number and page number.
- Two indexes: dense embeddings in a vector store, and a BM25 keyword index for exact matches on terms such as "PED", "48 months" or "Clause 4.2".

3.3 Retrieval pipeline

- Hybrid retrieval. Dense and BM25 results are fused with an ensemble retriever, since clause lookup is partly semantic ("knee replacement") and partly literal ("Clause 4.2").
- Cross-encoder re-ranking, to narrow the fused candidates down to a smaller and more precise set.
- Metadata filtering by policy. The policy is selected in the interface and applied as a filter during retrieval. Without it, the system can pull Insurer A's waiting period while judging Insurer B's policy and return a confident wrong answer.
- Query decomposition. One claim becomes several retrieval queries, one for each check that applies.

3.4 Agentic orchestration (LangGraph)

A state graph over a typed claim state, with five nodes:

1. Claim parser and planner. Extracts the structured claim (age, ailment, procedure, policy inception date, admission date, sum insured, room category) and lists the clause checks this claim needs.
2. Retriever. Runs the hybrid and re-rank pipeline for each check.
3. Relevance grader. Scores the retrieved evidence for each check. If it is weak, the node rewrites the query and tries again a limited number of times, and marks the check unresolved if that also fails.
4. Benefit calculator (tool node). Plain Python for elapsed months against the waiting period, proportionate room rent deduction, sub-limit capping and co-payment. The arithmetic is kept out of the LLM.
5. Adjudicator. Writes the verdict with a citation for each clause used, and returns "insufficient evidence, refer to human" if any required check is unresolved.

Conditional edges handle the retry and abstain paths, so the route taken through the graph depends on the claim.

Nodes that return structured data use a Pydantic output parser, and malformed output is re-prompted within the same retry limit. Smaller open-weight models do not always follow a required output format, and we would rather catch that as an error than let it through as a wrong verdict.

3.5 Evaluation

- RAGAS: faithfulness, context precision and context recall on the gold set.
- Task metrics: verdict accuracy and clause citation accuracy.
- Ablation: naive RAG (fixed-size chunks, dense retrieval only, single LLM call) compared against the full pipeline, metric by metric. We plan to report this comparison in the final presentation.

3.6 Interface

A web app built with Streamlit. The user selects a policy, enters a claim scenario, and sees the verdict, the payable calculation and the cited clauses next to their source page. The LangGraph state is shown after each run, so the path taken through the graph is visible during the demo.

4. Tech Stack

Layer	Choice
Orchestration	LangChain: document loaders, splitters, retrievers, chains, output parsers
Agent framework	LangGraph: StateGraph, conditional edges, tool node, bounded retry loop
LLM	Llama, served through the Groq API (free tier)
Embeddings	nomic-embed-text, served locally through Ollama
Vector store	Chroma
Keyword retrieval	BM25Retriever, fused with EnsembleRetriever
Re-ranking	Cross-encoder re-ranker (sentence-transformers), run locally
PDF parsing	pdfplumber, for page-level provenance
Evaluation	RAGAS, with a short accuracy script over our gold set
Interface	Streamlit
Tooling	Python, Git / GitHub

We are following the LangChain and LangGraph syntax used in class. BM25 fusion, cross-encoder re-ranking, RAGAS and Streamlit are additions on top of that, limited to the retrieval and evaluation parts of the system.

5. Planned Division of Work

Member	Ownership
Gundeep Singh	The LangGraph agent: claim parser and planner, relevance grader, and the retry and abstain control flow. The retrieval pipeline: hybrid fusion, metadata filtering and cross-encoder re-ranking.
Aditya Bakhale	Document ingestion: clause-aware chunking and the metadata schema. The dense and BM25 indexes. The benefit calculator tool.
Raghavendra Prasad Shetty	The gold evaluation set. The RAGAS harness and the ablation study. The Streamlit web app.

All three of us work on prompt design and review the evaluation. In the demo, Gundeep covers the architecture and the agent's execution path, Aditya covers retrieval and clause grounding, and Raghavendra presents the evaluation results.